

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería
Escuela de Ciencias y Sistemas

MANUAL TÉCNICO

API REST, vistas y triggers
SQL Server 2022 en Docker

Comercial La Estrella · Fase 2

Estudiante: Alex Ricardo Castañeda Rodríguez
Carné: 202300476
Curso: Bases de Datos 1
Período: Segundo semestre 2026
Fecha: 16 de octubre de 2026

Guatemala, 2026

Índice

1. Sistema implementado **3**
1.1. Distribución de los componentes . . . 3
1.2. Secuencia de atención de una operación . . . 3

2. Instalación y puesta en servicio **4**
2.1. Condiciones del entorno . . . 4
2.2. Preparación de la configuración . . . 4
2.3. Inicio del motor e inicialización . . . 4
2.4. Inicio y comprobación del backend . . . 5
2.5. Comprobación sobre la instalación en servicio . . . 5
2.6. Arranque de una instalación existente . . . 6

3. Persistencia y adaptación del modelo **6**
3.1. Claves y consistencia entre establecimientos . . . 6
3.2. Tipos y conservación de los datos . . . 7

4. Carga histórica y conciliación **7**
4.1. Procedimiento de carga inicial . . . 7
4.2. Control del archivo y exclusión de cargas simultáneas . . . 7
4.3. Conservación del inventario histórico . . . 7
4.4. Conciliación antes de confirmar . . . 8

5. Operación del servicio REST **8**
5.1. Alta, consulta, modificación y eliminación . . . 8
5.2. Comprobación del ciclo de actualización y eliminación . . . 9
5.3. Validación aplicada . . . 10
5.4. Rutas por entidad . . . 10

6. Gestión del inventario y registro del detalle **11**
6.1. Ajuste aplicado por Insert_Detalle . . . 11
6.2. Resultado de una venta válida y de un rechazo . . . 11
6.3. Petición rechazada y resultado conservado . . . 11
6.4. Lotes y solicitudes concurrentes . . . 13

7. Registro de pagos y conciliación de la venta **14**
7.1. Actualización realizada por Insert_Pago . . . 14
7.2. Secuencia comprobada con abonos . . . 14
7.3. Abonos ejecutados sobre la misma venta . . . 14
7.4. Correcciones y protección de movimientos . . . 15

8. Cambios de estado, anulación y protección del histórico **16**

9. Consultas analíticas **16**

9.1. Ventas por ubicación . . . 16
9.2. Productos vendidos . . . 16
9.3. Saldos y consulta consolidada de ventas . . . 16
9.4. Resultados consultados en el servidor . . . 17

10. Uso de las colecciones de Postman **18**

10.1. Preparación de cada ejecución . . . 19
10.2. Interpretación de los resultados . . . 19

11. Acceso a datos y control de errores **19**

12. Verificación de la instalación y mantenimiento **19**

12.1. Comprobaciones automatizadas . . . 19
12.2. Persistencia . . . 20
12.3. Diagnóstico operativo . . . 20
12.4. Inspección y resguardo . . . 20

A. Implementación de las consultas y controles transaccionales **21**

A.1. Vista_Ventas_Ubicacion . . . 21
A.2. Vista_Top_Productos . . . 21
A.3. Vista_Consulta_Ventas . . . 22
A.4. Vista_Saldos_Pendientes . . . 22
A.5. Insert_Detalle . . . 23
A.6. Insert_Pago . . . 24
A.7. Validar_Venta . . . 25
A.8. Proteger_Catalogo . . . 25
A.9. Proteger_Estados . . . 26

B. Referencias de la implementación **26**

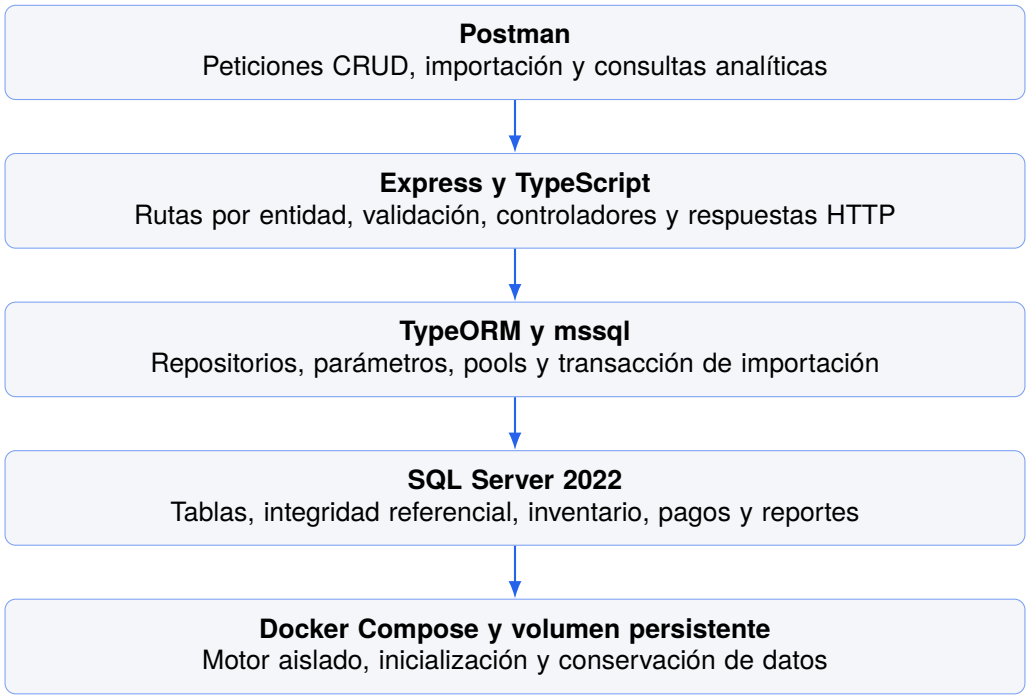
1. Sistema implementado

Comercial La Estrella administra el inventario de productos por tienda, el registro de ventas y sus abonos. La segunda fase traslada el modelo de 19 tablas a SQL Server 2022 y expone sus operaciones mediante una API REST. La instalación incluye el motor en Docker, la inicialización automática de su estructura, el servidor Express y las colecciones de Postman utilizadas para operar y verificar el sistema.

La estructura de datos conserva las claves del proyecto anterior. Las existencias se gestionan por combinación de tienda y producto; el detalle conserva el precio aplicado al vender y los pagos se acumulan por venta. Las reglas de inventario y de estado se ejecutan en SQL Server, de manera que una petición HTTP no puede confirmar un movimiento que incumpla esas reglas.

La operación se realiza desde Postman. La consulta estructural de tablas, restricciones, vistas y triggers puede efectuarse con DBeaver. La solución entregada no incluye frontend: las operaciones solicitadas se exponen y demuestran mediante los endpoints del backend.

1.1. Distribución de los componentes



1.2. Secuencia de atención de una operación

Express recibe el JSON y selecciona el módulo de rutas de la entidad. La validación comprueba campos, tipos y claves antes de llegar al repositorio. TypeORM ejecuta la escritura con valores parametrizados. SQL Server evalúa las restricciones y los triggers dentro de la misma operación; si se produce un rechazo, la API comunica el código y el mensaje correspondientes. Después de una escritura correcta, el servidor consulta el registro resultante para devolver su estado efectivo.

En una venta, esta secuencia es especialmente importante: el resultado del POST de detalle depende de las existencias de la tienda y el estado posterior de la venta depende de los pagos efectivamente almacenados. El backend no mantiene una copia del stock ni calcula un estado independiente del motor.

2. Instalación y puesta en servicio

2.1. Condiciones del entorno

Se utiliza Docker con contenedores Linux y el complemento Docker Compose, Node.js 22 o superior, npm y Python 3 para preparar la configuración. Postman se utiliza como cliente del servicio. Los puertos predeterminados son 1433 para SQL Server y 3000 para la API. Ambos se publican en la interfaz local.

SQL Server tiene un presupuesto interno configurado de 2048 MB. Además del motor, el equipo debe disponer de memoria para Node, Postman y el sistema operativo. Ese valor no constituye un límite de memoria del contenedor. Las dependencias se instalan desde package-lock.json para conservar la versión del árbol utilizado por el proyecto.

2.2. Preparación de la configuración

Los comandos se ejecutan en la raíz de `Proyecto 2`:

```
python3 scripts/setup-env.py
npm ci
```

El primer comando crea .env a partir de la plantilla y asigna secretos aleatorios. No reemplaza un archivo existente. La configuración distingue el usuario administrativo del contenedor, el usuario normal del backend y el usuario exclusivo de la carga histórica.

- `DB_HOST`, `DB_PORT` y `DB_NAME`: servidor, puerto y base ComercialLaEstrella.
- `DB_USER` y `DB_PASSWORD`: credenciales del login estrella_api.
- `IMPORT_DB_USER` y `IMPORT_DB_PASSWORD`: credenciales de estrella_import.
- `IMPORT_ENABLED`: false durante la operación habitual.
- `HOST` y `PORT`: dirección y puerto de escucha HTTP.

El archivo local de credenciales no forma parte del repositorio ni de la entrega. Modificar una contraseña en .env no actualiza un login ya creado en el volumen: la contraseña del motor y la del archivo deben mantenerse coordinadas.

2.3. Inicio del motor e inicialización

```
docker compose up -d
docker compose ps -a
docker compose logs db-init
```

El servicio sqlserver inicia SQL Server 2022 y conserva sus archivos en el volumen del proyecto. La imagen está fijada por digest. La comprobación de salud ejecuta una consulta al motor; después de que sea satisfactoria, db-init ejecuta init.sql mediante sqlcmd.

La instalación se considera preparada cuando el motor está saludable y db-init ha terminado con código 0. El inicializador crea ComercialLaEstrella, las tablas ausentes, las restricciones, los índices, las vistas, los triggers y los usuarios. No inserta los registros del histórico. Las vistas y los triggers se definen con CREATE OR ALTER, mientras que las tablas y los logins se crean únicamente si no existen.

Para reaplicar la inicialización sobre el esquema previsto sin eliminar datos:

```
docker compose run --rm db-init
```

La repetición del script no constituye una migración automática de tablas que hayan cambiado de definición. Cualquier modificación estructural posterior requiere revisar el DDL antes de aplicarla a una base existente.

2.4. Inicio y comprobación del backend

```
npm run build
npm start
```

El servidor establece su conexión SQL antes de abrir el puerto HTTP. Si no consigue autenticarse o encontrar la base, no inicia el servicio normalmente. La terminal permanece abierta mientras la API está disponible. El cierre con Ctrl+C detiene la escucha y libera los pools.

La comprobación inicial se realiza con GET /api/health. El servidor ejecuta SELECT 1; la respuesta satisfactoria es HTTP 200 con estado ok y database SQL Server 2022. Esta consulta permite distinguir un backend activo y conectado de un proceso que únicamente tiene abierto el puerto.

2.5. Comprobación sobre la instalación en servicio

Las capturas se obtuvieron directamente en un navegador que consultó la API local. Muestran el JSON servido por el backend, con el formato de lectura del navegador. No se reconstruyeron las respuestas en una página de presentación ni se simula la interfaz de Postman. La ejecución y sus códigos HTTP se conservaron en el registro de operaciones asociado.

Disponibilidad y estado de la carga



Figura 1: GET /api/health sobre el servidor activo: estado ok y conexión con SQL Server 2022.

Pretty-print ☑


```
{
  "importado": true,
  "detalle": {
    "fecha": "2026-10-17T00:16:57.233Z",
    "filas": 9068,
    "sha256": "80e4d18ae9300bcedbdc1354eddb17b29d2415e053085a9e05d0fae6462882ab"
  },
  "habilitada": false
}
```


Figura 2: GET /api/admin/importacion: histórico confirmado, 9068 filas originales y ruta de carga deshabilitada.

2.6. Arranque de una instalación existente

Cuando el histórico ya está cargado, se inicia Compose, se verifica el motor y se ejecuta npm start con IMPORT_ENABLED=false. No se vuelve a importar ni a generar credenciales. Si se ha modificado TypeScript, se compila antes de iniciar; para desarrollo se dispone de npm run dev.

Los reinicios conservan el volumen. docker compose stop y docker compose start permiten detener y recuperar los servicios. docker compose down conserva los datos; down -v elimina el volumen y no se utiliza como procedimiento de reinicio.

3. Persistencia y adaptación del modelo

El modelo se organiza en ubicación, catálogos comerciales, personas y establecimientos, productos e inventario, y transacciones. pais, departamento y municipio establecen la jerarquía territorial. persona contiene los datos compartidos por cliente y empleado; tienda identifica el establecimiento donde se realiza la venta.

producto conserva descripción, categoría y marca. catalogo_producto añade el precio vigente y la existencia de ese producto en una tienda concreta. Esta separación permite que un mismo producto tenga distinta disponibilidad y precio en cada establecimiento. La venta identifica tienda, empleado, cliente, fecha y estado; detalle_venta conserva cantidades y precios históricos; pago registra cada abono y su método.

3.1. Claves y consistencia entre establecimientos

catalogo_producto utiliza la PK compuesta id_tienda, id_producto. detalle_venta utiliza id_venta, id_producto. Estas claves impiden duplicar la misma asociación y se conservan en el orden de los segmentos HTTP.

La FK compuesta de venta sobre empleado(id_empleado,id_tienda) impide registrar una operación en una tienda distinta de la asignada al empleado. La integridad no depende de que Postman seleccione correctamente el empleado: el motor rechaza la combinación inválida.

3.2. Tipos y conservación de los datos

Los identificadores se mantienen explícitos y no utilizan IDENTITY. Los valores NUMBER de la fase anterior se expresan como INT o DECIMAL(12,2); los textos se almacenan como NVARCHAR y las fechas como DATETIME2(0). Los valores de fecha se interpretan en UTC para evitar desplazamientos durante la carga y la lectura.

Los 45 correos ausentes del histórico se conservan como NULL. El índice único filtrado de persona incluye únicamente correos no nulos; así se permiten varios registros sin correo y se conserva la unicidad de los correos presentes. Los importes deben ser positivos, las existencias no negativas y el subtotal debe coincidir con cantidad por precio unitario.

4. Carga histórica y conciliación

La carga inicial procede de las 19 hojas del dataset de la fase 1. Se entrega su conversión JSON y un manifiesto de conteos. El proceso conserva identificadores, importes, fechas, estados y existencias originales. El resumen de filas se presenta al final de esta sección.

4.1. Procedimiento de carga inicial

La importación solo se realiza sobre las 19 tablas de negocio vacías. Se detiene la API, se habilita temporalmente IMPORT_ENABLED=true y se inicia de nuevo. Desde otra terminal se ejecuta:

```
npm run import:historico
```

El cliente envía POST /api/admin/importacion con el archivo completo y la cabecera X-Import-Key. La respuesta correcta es HTTP 201, mensaje Historico importado y conciliado y total 9068. Terminada la carga, se restaura IMPORT_ENABLED=false y se reinicia el backend. GET /api/admin/importacion debe confirmar importado=true y detalle.filas=9068.

La colección separada de importación permite enviar el mismo archivo desde Postman mediante Body en modo binary. La clave se define localmente. Esta colección no interviene en las ejecuciones habituales de CRUD y triggers.

4.2. Control del archivo y exclusión de cargas simultáneas

El servicio compara la representación canónica del cuerpo con el archivo original mediante SHA-256 y valida cada campo con las reglas de la entidad. Un archivo modificado se rechaza antes de escribir. La exclusión en memoria se reserva antes de la primera lectura asíncrona; dentro de SQL se adquiere además un bloqueo exclusivo de aplicación asociado a la transacción.

La transacción SERIALIZABLE consulta el registro de importación y comprueba que las tablas están vacías. La carga se realiza en orden de dependencias y en bloques de 50 filas. Una segunda carga completa se rechaza con 409; una ruta deshabilitada responde 404 y una clave incorrecta, cuando está habilitada, responde 401.

4.3. Conservación del inventario histórico

Las existencias del dataset ya representan su situación histórica. Descontar nuevamente todos sus detalles produciría un saldo distinto del original. Para esta única carga se utiliza estrella_import, miembro del rol importador, y un contexto de sesión temporal. Los triggers operativos reconocen la combinación de ambas condiciones para omitir el descuento y el recálculo del estado durante la importación.

Las PK, FK, CHECK e índices siguen activos. El login normal de la API no pertenece al rol y no puede habilitar esa excepción únicamente asignando el contexto. La excepción tampoco permite introducir un archivo arbitrario: la comparación previa exige el contenido original.

4.4. Conciliación antes de confirmar

El servicio vuelve a leer todas las tablas, ordena por clave y compara todos sus valores con el contenido de origen. Las fechas se normalizan antes de comparar. Si difiere un valor o falla una sentencia, se revierte la carga completa. Solo después de conciliar se registra fecha, número de filas y hash y se confirma la transacción. Finalmente se limpia el contexto y se destruye el pool especial, incluso en una salida por error.

El total 9068 corresponde a esa carga, no al conteo permanente de la base. Las operaciones de prueba posteriores pueden añadir filas sin cambiar el registro de importación.

Tabla	Filas originales	Clave primaria
pais	3	id_pais
departamento	32	id_departamento
municipio	80	id_municipio
tipo_tienda	4	id_tipo_tienda
tipo_identificacion	3	id_tipo_identificacion
cargo	4	id_cargo
categoria	10	id_categoria
marca	15	id_marca
estado_venta	3	id_estado_venta
metodo_pago	4	id_metodo_pago
persona	388	id_persona
tienda	20	id_tienda
empleado	88	id_empleado
cliente	300	id_cliente
producto	114	id_producto
catalogo_producto	1,424	id_tienda, id_producto
venta	1,500	id_venta
detalle_venta	3,687	id_venta, id_producto
pago	1,389	id_pago
Total	**9,068**	19 tablas de negocio

5. Operación del servicio REST

La base de las peticiones es http://127.0.0.1:3000/api. POST y PUT utilizan Content-Type application/json. La API admite campos declarados por entidad; un campo desconocido, una clave primaria en PUT o un objeto de actualización vacío se rechazan antes de la escritura.

5.1. Alta, consulta, modificación y eliminación

POST /api/entidad crea un registro con sus identificadores explícitos y devuelve 201 con el objeto almacenado. GET del listado devuelve data, total, limit y offset. GET de una clave devuelve el registro o 404. PUT modifica uno o varios campos permitidos y devuelve el registro actualizado. DELETE devuelve 204 sin cuerpo; las referencias o protecciones de negocio pueden impedirlo con 409.

Los listados se ordenan por PK. limit admite 1 a 500 y usa 100 por defecto; offset comienza en cero. Para recorrer una tabla grande se conserva el límite y se aumenta offset. El valor total indica las filas de la entidad y no solo las de la página.

5.2. Comprobación del ciclo de actualización y eliminación

La operación registrada creó un país auxiliar, modificó su nombre mediante PUT y consultó el objeto actualizado. Después lo eliminó con DELETE y verificó la ausencia mediante GET. El registro de la ejecución confirmó 201, 200, 204 y 404 en los pasos correspondientes. Las capturas muestran el contenido obtenido antes y después de la eliminación.

Actualización y eliminación verificadas

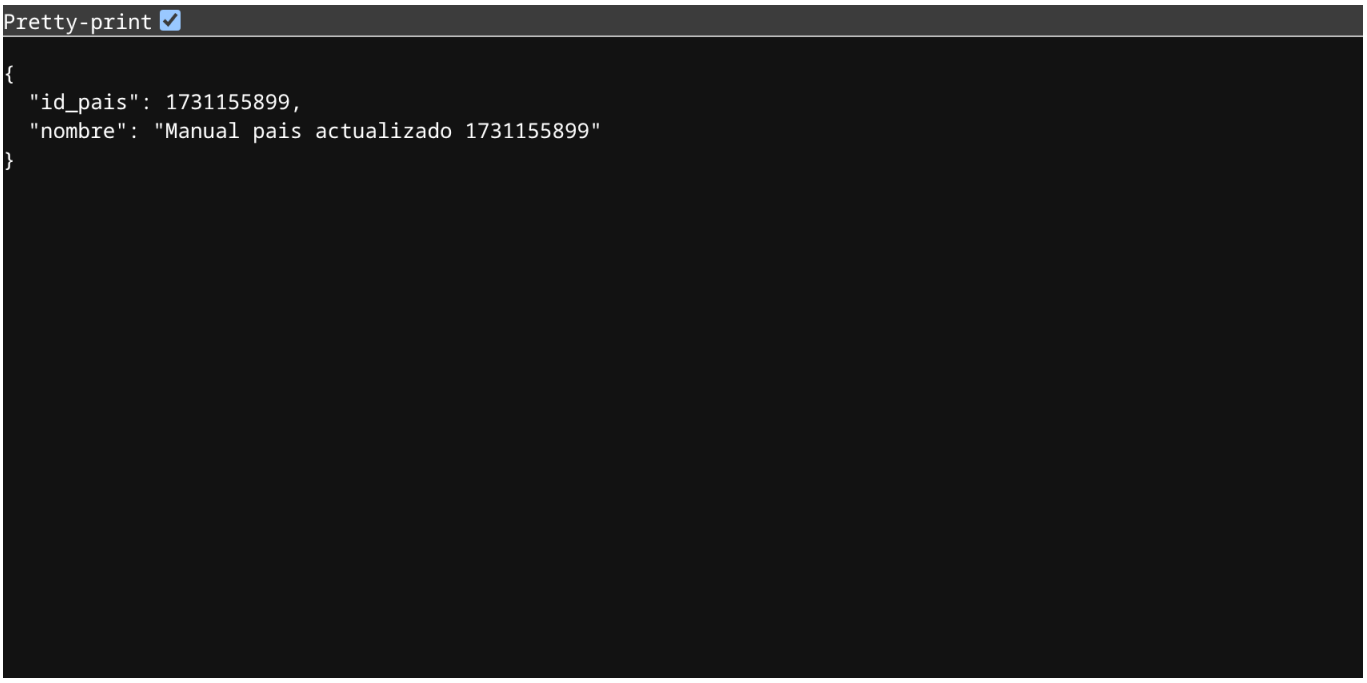


Figura 3: GET /api/pais/1731155899: nombre actualizado conservado en el registro de prueba.

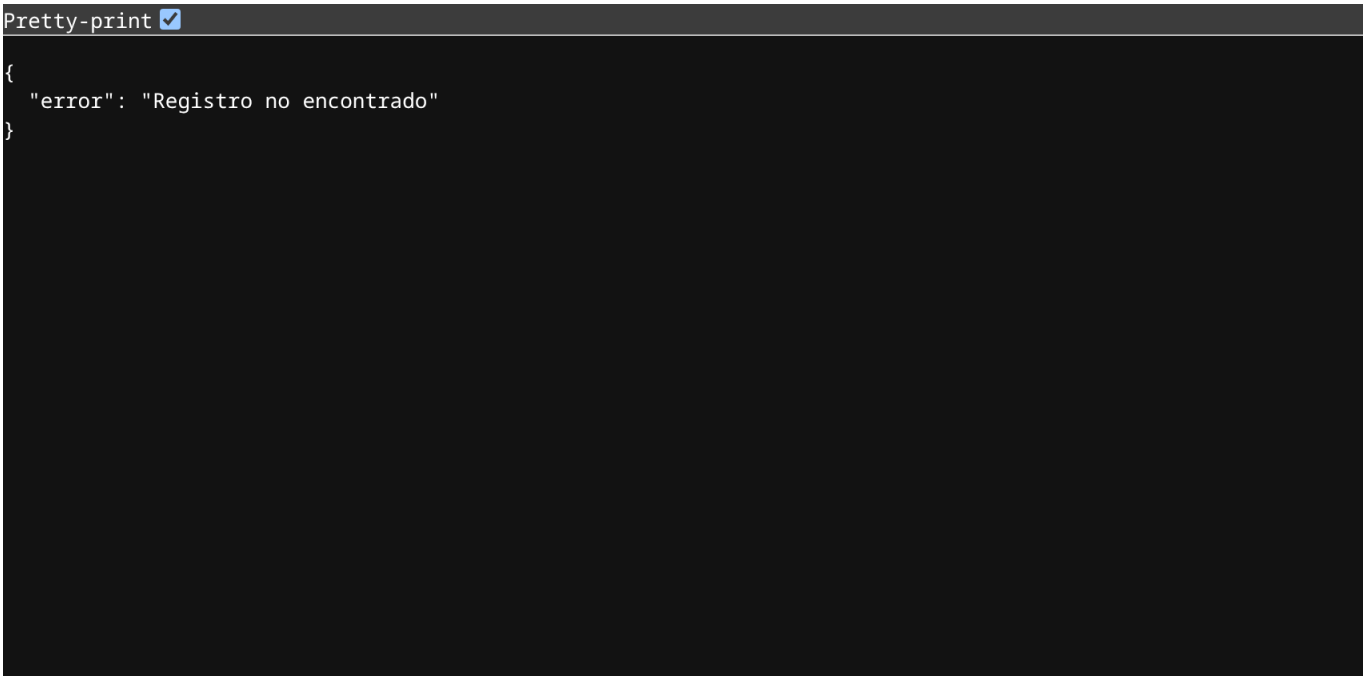


Figura 4: GET /api/pais/1731155899 después de DELETE: Registro no encontrado, correspondiente al HTTP 404 registrado.

5.3. Validación aplicada

Los IDs y las cantidades se envían como números enteros positivos; existencia_actual admite cero. Los importes se envían como números JSON con hasta dos decimales. Las fechas aceptan YYYY-MM-DD o YYYY-MM-DDTHH:mm:ss; el servidor puede devolverlas con Z al representarlas en JSON. correo admite una dirección válida o NULL. Las longitudes de texto corresponden al esquema SQL.

En pago, id_venta no puede cambiarse mediante PUT. En venta, las restricciones adicionales dependen del estado y de la existencia de detalles o pagos. Los errores de integridad se traducen a respuestas HTTP sin exponer consultas completas ni credenciales.

5.4. Rutas por entidad

Los listados y altas utilizan el nombre de la entidad; las operaciones sobre un registro añaden las claves de esta tabla. Las rutas de lote preceden a las rutas individuales para evitar interpretar lote como un ID.

Entidad	Ruta individual a partir de /api
pais	/pais/:id_pais
departamento	/departamento/:id_departamento
municipio	/municipio/:id_municipio
tipo_tienda	/tipo_tienda/:id_tipo_tienda
tipo_identificacion	/tipo_identificacion/:id_tipo_identificacion
cargo	/cargo/:id_cargo
categoria	/categoria/:id_categoria
marca	/marca/:id_marca
estado_venta	/estado_venta/:id_estado_venta
metodo_pago	/metodo_pago/:id_metodo_pago

Entidad	Ruta individual a partir de /api
persona	/persona/:id_persona
tienda	/tienda/:id_tienda
empleado	/empleado/:id_empleado
cliente	/cliente/:id_cliente
producto	/producto/:id_producto
catalogo_producto	/catalogo_producto/:id_tienda/:id_producto
venta	/venta/:id_venta
detalle_venta	/detalle_venta/:id_venta/:id_producto
pago	/pago/:id_pago

6. Gestión del inventario y registro del detalle

El alta de un detalle requiere una venta REGISTRADA sin pagos y un catálogo del producto en la tienda de esa venta. El precio unitario se conserva en el detalle; no se recalcula a partir del precio vigente al consultar operaciones históricas. cantidad debe ser positiva y subtotal igual a cantidad multiplicada por precio_unitario.

6.1. Ajuste aplicado por Insert_Detalle

Insert_Detalle reúne las ventas afectadas, bloquea sus encabezados y verifica que admitan cambios. Después agrupa las filas nuevas y anteriores por tienda y producto. El ajuste utilizado es la suma de cantidades nuevas menos la suma de cantidades anteriores.

Con una inserción de dos unidades se descuentan dos. Al modificar de dos a tres se descuenta solo una adicional. Al eliminar ese detalle se devuelven las tres unidades. La operación se calcula por conjunto de filas, por lo que también cubre los lotes.

La actualización del catálogo exige existencias suficientes y usa UPDLOCK y HOLDLOCK. El trigger comprueba que se hayan actualizado todos los grupos del movimiento. Si falta un catálogo o no alcanza alguna existencia, lanza el error 51001; la API devuelve 400 con Stock insuficiente. Se revierten tanto el detalle como los descuentos efectuados por esa sentencia.

6.2. Resultado de una venta válida y de un rechazo

La comprobación realizada para este manual utiliza un producto nuevo, existencia 10 y una venta registrada en tienda 1. Una solicitud de 11 unidades se rechaza y la consulta posterior conserva existencia 10; tampoco existe el detalle rechazado. Una solicitud de dos unidades se acepta y deja existencia 8. Las capturas de la sección de resultados corresponden al GET de la API después de cada operación.

6.3. Petición rechazada y resultado conservado

Operación ejecutada: `POST/api/detalle_venta`


```
{
  "id_venta": 1731155899,
  "id_producto": 1731155899,
  "cantidad": 11,
  "precio_unitario": 25,
  "subtotal": 275
}
```

Respuesta recibida: HTTP 400


```
{
  "error": "Stock insuficiente"
}
```


Después del rechazo se consultó también el detalle y se obtuvo 404. Esto confirma que la transacción no dejó registrado el movimiento que intentaba vender 11 unidades. La siguiente petición utilizó cantidad 2, precio 25 y subtotal 50; respondió 201 y permitió observar el descuento a 8.

Existencias antes y después de las operaciones

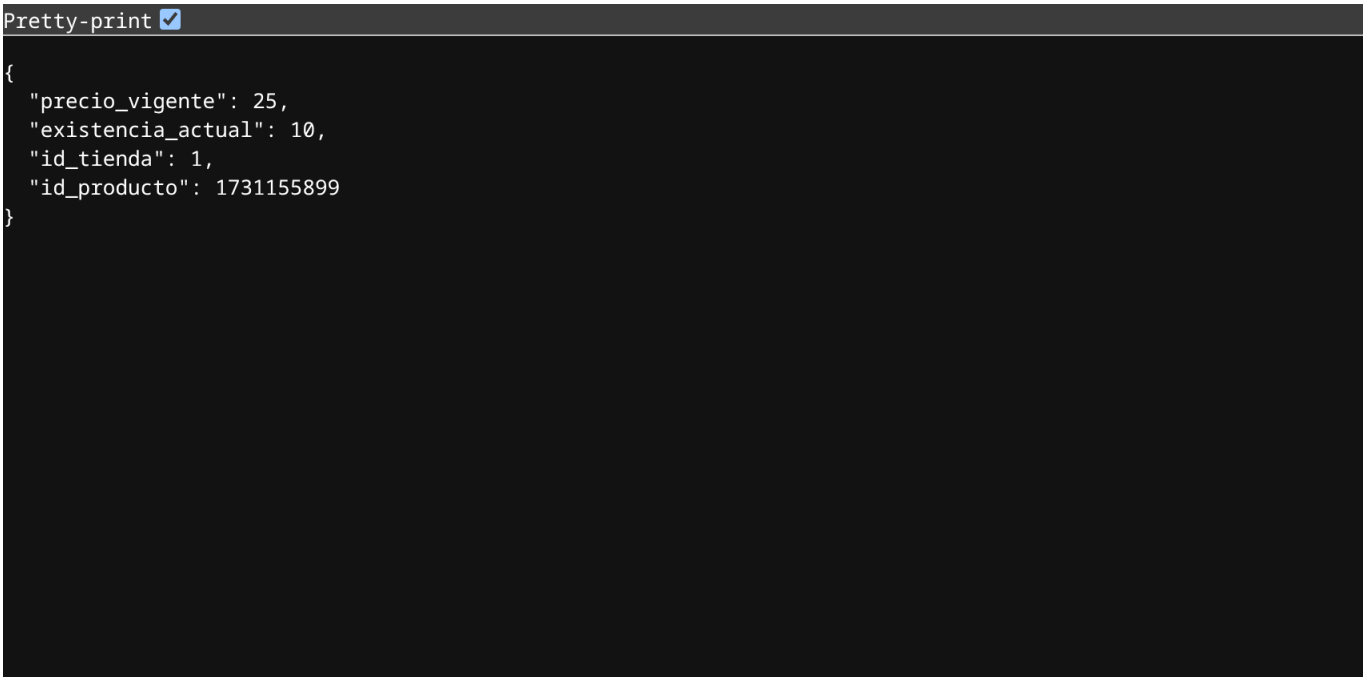


Figura 5: Catálogo del producto 1731155899 en tienda 1, antes de registrar detalles: existencia_actual 10.

Pretty-print ☑


```
{
  "precio_vigente": 25,
  "existencia_actual": 10,
  "id_tienda": 1,
  "id_producto": 1731155899
}
```


Figura 6: Consulta posterior al POST rechazado por stock: se conservan las 10 unidades.

Pretty-print ☑


```
{
  "precio_vigente": 25,
  "existencia_actual": 8,
  "id_tienda": 1,
  "id_producto": 1731155899
}
```


Figura 7: Consulta posterior al POST válido de dos unidades: existencia_actual 8.

6.4. Lotes y solicitudes concurrentes

POST /api/detalle_venta/lote admite de 1 a 100 objetos. La inserción se envía como una sola sentencia. Si dos filas del lote consumen el mismo producto de una tienda, el trigger valida su suma; no evalúa cada fila como si dispusiera del stock inicial por separado. Si una falla, el lote no conserva cambios parciales.

Las solicitudes concurrentes también se resuelven en el motor. La condición de disponibilidad y los bloqueos protegen

las mismas existencias frente a operaciones que llegan simultáneamente. Los conflictos de bloqueo clasificados por el middleware responden 409 y pueden requerir reintento. La prueba de concurrencia verifica que no se vendan las mismas últimas unidades dos veces.

7. Registro de pagos y conciliación de la venta

Cada pago almacena venta, método e importe. La venta debe tener detalles y no estar anulada. El pago se registra con un ID propio y no se traslada posteriormente a otra venta. La corrección de su monto o su eliminación sí provoca un nuevo cálculo del estado.

7.1. Actualización realizada por Insert_Pago

El trigger identifica las ventas afectadas y bloquea sus encabezados. Obtiene por separado la suma de subtotales y la suma de abonos. Si el total de la venta es mayor que cero y los abonos lo cubren o superan, asigna PAGADA; en otro caso mantiene o restablece REGISTRADA.

La implementación acepta un importe abonado superior al total. En ese caso el saldo calculado puede ser negativo. El comportamiento no debe confundirse con un rechazo por sobrepago: el criterio del estado es cobertura del total, no igualdad estricta.

7.2. Secuencia comprobada con abonos

En la venta de dos unidades a Q25 el total es Q50. El primer pago de Q20 conserva REGISTRADA, total_pagado=20 y saldo_pendiente=30. El segundo pago de Q30 hace que SQL Server asigne PAGADA, con total_pagado=50 y saldo_pendiente=0. El resultado se obtiene mediante GET de la venta, que consulta la vista auxiliar con los importes consolidados.

7.3. Abonos ejecutados sobre la misma venta

Primer abono enviado por HTTP:

```
{
  "id_pago": 1731155899,
  "id_venta": 1731155899,
  "id_metodo_pago": 1,
  "monto": 20
}
```

El segundo abono conserva la venta y utiliza un ID de pago distinto y monto 30. Ambos POST respondieron 201. Las consultas siguientes corresponden a esa misma venta y permiten comprobar que el estado cambió por el registro de los abonos, sin un PUT manual que asignara PAGADA.

Conciliación observada en la API

Pretty-print


```
{
  "id_venta": 1731155899,
  "fecha_venta": "2026-10-16T12:00:00.000Z",
  "id_tienda": 1,
  "id_empleado": 1,
  "id_cliente": 1,
  "id_estado_venta": 1,
  "estado": "REGISTRADA",
  "total_venta": 50,
  "total_pagado": 20,
  "saldo_pendiente": 30
}
```


Figura 8: GET /api/venta/1731155899 después del abono de Q20: REGISTRADA, total 50, pagado 20 y saldo 30.

Pretty-print


```
{
  "id_venta": 1731155899,
  "fecha_venta": "2026-10-16T12:00:00.000Z",
  "id_tienda": 1,
  "id_empleado": 1,
  "id_cliente": 1,
  "id_estado_venta": 2,
  "estado": "PAGADA",
  "total_venta": 50,
  "total_pagado": 50,
  "saldo_pendiente": 0
}
```


Figura 9: La misma venta después del abono restante de Q30: PAGADA, total_pagado 50 y saldo_pendiente 0.

7.4. Correcciones y protección de movimientos

Reducir o eliminar un abono puede devolver la venta a REGISTRADA cuando los pagos restantes no cubren el total. Los detalles no se modifican mientras existen pagos, evitando que una venta abonada cambie de importe por una

edición del detalle. Una venta sin detalles no recibe pagos y una venta anulada tampoco admite nuevos abonos.

8. Cambios de estado, anulación y protección del histórico

Las ventas nuevas comienzan REGISTRADA. Validar_Venta bloquea el cambio manual a PAGADA y permite el cambio interno que realiza Insert_Pago. Cuando existen detalles o pagos, la tienda, el empleado, el cliente y la fecha dejan de ser modificables: cambiar esos datos alteraría la asociación de movimientos ya registrados.

Una venta REGISTRADA sin pagos puede anularse. El trigger agrupa las cantidades de sus detalles y devuelve el inventario a cada catálogo. La devolución ocurre únicamente en la transición al estado ANULADA. Un intento posterior de repetir la transición se rechaza, evitando devolver las unidades más de una vez. La venta anulada queda protegida y se excluye de los reportes operativos.

Proteger_Catalogo impide borrar un catálogo que da contexto a detalles existentes. Las FK impiden borrar ventas con detalles o pagos asociados. No se configura borrado en cascada de esos movimientos. Los estados del sistema se mantienen fijos: 1 REGISTRADA, 2 PAGADA y 3 ANULADA; su modificación o eliminación se rechaza.

9. Consultas analíticas

Los endpoints de reportes consultan las vistas definidas en init.sql. Las agregaciones se ejecutan en SQL Server y el backend aplica un orden de presentación. Las respuestas contienen data como arreglo; los valores reflejan los datos presentes al momento de la consulta, incluidos los movimientos de prueba que no se hayan eliminado.

9.1. Ventas por ubicación

Vista_Ventas_Ubicacion relaciona la venta con tienda, municipio, departamento y país. Antes de agrupar por tienda obtiene el total de cada venta desde sus detalles; así el número de ventas no se multiplica por la cantidad de productos de cada una. Excluye ventas ANULADA y devuelve cantidad_ventas y total_facturado junto con la ubicación.

La vista parte de las ventas existentes. Una tienda sin ventas no aparece solamente por existir en el catálogo de tiendas. El endpoint /api/reportes/ventas-ubicacion ordena por id_tienda.

9.2. Productos vendidos

Vista_Top_Productos agrupa los detalles de ventas no anuladas por producto, categoría y marca. Suma unidades e importe vendido y conserva el código del producto. El endpoint /api/reportes/top-productos ordena por unidades descendentes y código como desempate; devuelve todos los productos presentes en la agregación, no únicamente diez.

9.3. Saldos y consulta consolidada de ventas

Vista_Consulta_Ventas preagrupa los detalles y los pagos en dos conjuntos distintos, y después los relaciona con el encabezado. Una unión directa de varios detalles con varios abonos repetiría los importes. Al agregarlos antes, cada venta recibe un único total vendido y un único total pagado. COALESCE representa los conjuntos ausentes con cero.

El saldo se calcula como total_venta menos total_pagado. Vista_Saldos_Pendientes selecciona de esa consulta las ventas REGISTRADA. El endpoint /api/reportes/saldos-pendientes devuelve sus IDs e importes. Los GET de venta utilizan la vista auxiliar para mostrar también estado y saldos. Algunos agregados BIGINT pueden aparecer como cadenas JSON en la respuesta del controlador.

9.4. Resultados consultados en el servidor

Las capturas siguientes muestran el inicio de cada respuesta JSON obtenida directamente del endpoint. Se visualiza un extracto por el tamaño de la ventana; el registro de la ejecución conserva los arreglos completos. Las cifras corresponden al estado de la base tras las pruebas y no deben interpretarse como totales exclusivos del archivo histórico.

Consultas de las vistas

Pretty-print ☑


```
{
  "data": [
    {
      "id_tienda": 1,
      "tienda": "Comercial La Estrella Zona 1",
      "municipio": "Fray Bartolome de las Casas",
      "departamento": "Alta Verapaz",
      "pais": "Guatemala",
      "cantidad_ventas": "77",
      "total_facturado": 76797.89
    },
    {
      "id_tienda": 2,
      "tienda": "Comercial La Estrella Mixco",
      "municipio": "Chimaltenango",
      "departamento": "Chimaltenango",
      "pais": "Guatemala",
      "cantidad_ventas": "63",
      "total_facturado": 64755.5
    },
    {
      "id_tienda": 3
```


Figura 10: GET /api/reportes/ventas-ubicacion: extracto de tiendas con ubicación, cantidad de ventas e importe facturado.

Pretty-print


```
{
  "data": [
    {
      "codigo_producto": 90,
      "nombre": "Helado de vainilla Light",
      "categoria": "Congelados",
      "marca": "GranjaFeliz",
      "unidades_vendidas": "174",
      "monto_total": 10461.93
    },
    {
      "codigo_producto": 19,
      "nombre": "Tortillas de maiz frito Especial",
      "categoria": "Snacks",
      "marca": "La Espiga",
      "unidades_vendidas": "162",
      "monto_total": 9623.54
    },
    {
      "codigo_producto": 41,
      "nombre": "Pan de molde Light",
```


Figura 11: GET /api/reportes/top-productos: extracto ordenado por unidades vendidas; incluye categoría, marca e importe.

Pretty-print


```
{
  "data": [
    {
      "id_venta": 12,
      "total_venta": 1246.2,
      "total_pagado": 0,
      "saldo_pendiente": 1246.2
    },
    {
      "id_venta": 16,
      "total_venta": 1422.6,
      "total_pagado": 0,
      "saldo_pendiente": 1422.6
    },
    {
      "id_venta": 18,
      "total_venta": 1285.5,
      "total_pagado": 632.32,
      "saldo_pendiente": 653.18
    },
    {
```


Figura 12: GET /api/reportes/saldos-pendientes: extracto de ventas registradas con total, pagado y saldo calculado.

10. Uso de las colecciones de Postman

Se importan ComercialLaEstrella.postman_collection.json y local.postman_environment.json. El primero crea Comercial La Estrella - Fase 2 en Collections; el segundo crea Comercial La Estrella local en Environments. Debe seleccionarse el entorno antes de enviar las peticiones.

La colección se ejecuta en el orden Preparacion, CRUD por entidad, Limpieza auxiliares, Triggers y Vistas. La preparación crea las dependencias que necesitan las entidades posteriores; la limpieza elimina los auxiliares del CRUD respetando sus referencias. Las pruebas de triggers mantienen movimientos identificables para consultar sus resultados.

10.1. Preparación de cada ejecución

baseUrl toma la dirección del servidor. runId, auxId y payId se generan en el script Pre-request de Iniciar ejecucion y comprobar conexion. Se conserva ese conjunto de identificadores durante toda la ejecución. Repetir un POST Crear sin regenerarlos vuelve a intentar la misma PK y puede responder 409.

Para una nueva sesión se ejecuta una vez esa primera petición y se continúa en orden. No borra registros, no reinicia el backend y no repite la carga histórica. Tampoco se debe ejecutar a mitad de un CRUD: las consultas siguientes pasarían a referirse a otros IDs. El rango aleatorio reduce colisiones, pero no garantiza que nunca coincidan con una ejecución anterior.

10.2. Interpretación de los resultados

El CRUD habitual debe producir 201 al crear, 200 al listar, buscar y actualizar, 204 al eliminar y 404 al confirmar que ya no existe. En estado_venta se esperan 409 en las escrituras protegidas. En inventario se espera 400 al pedir más unidades que las disponibles. En cada caso se verifica la aserción de Test Results y el cuerpo, no solamente el color del código HTTP.

Run collection con una iteración ejecuta la secuencia completa. Las carpetas aisladas requieren haber preparado antes las dependencias correspondientes. La colección de importación se reserva para la carga inicial de una base vacía y no forma parte de este recorrido habitual.

11. Acceso a datos y control de errores

El backend utiliza un módulo de rutas por entidad y un controlador común construido desde los metadatos declarados. Los campos del JSON y sus restricciones se validan con Zod. EntitySchema conserva nombres, claves y tipos SQL; las relaciones físicas se mantienen en el DDL. synchronize y migrationsRun permanecen desactivados para que el arranque del ORM no reescriba la estructura.

El pool habitual permite hasta cinco conexiones y el de importación una. disableOutputReturning evita que el ORM genere OUTPUT sin INTO en escrituras sobre tablas con triggers. Después de insertar o actualizar, el controlador consulta el registro y devuelve el resultado definitivo.

El error SQL 51001 se traduce a 400 con Stock insuficiente. Los rechazos de negocio 51002 se comunican como 409. Los duplicados 2601 y 2627, las referencias 547 y los conflictos de bloqueo clasificados también se comunican como 409. La entrada inválida responde 400; un recurso inexistente, 404; y un cuerpo que supera el límite de 10 MB, 413. Los errores no clasificados responden 500 sin divulgar consultas o credenciales.

12. Verificación de la instalación y mantenimiento

12.1. Comprobaciones automatizadas

```
npm test
npm run test:integration
npm run postman
```

Las pruebas de integración y Postman requieren la API iniciada y el histórico cargado. Realizan escrituras por HTTP y dejan algunos escenarios de demostración. La evidencia de la ejecución inicial registra cuatro pruebas unitarias, ocho escenarios de integración y 144 peticiones de Postman con 155 aserciones aprobadas.

La cobertura incluye CRUD de entidades relacionadas, rechazos de entrada, ajustes de inventario por inserción, actualización y eliminación, abonos y correcciones, lotes atómicos, competencia por existencias, anulación y comparación de las vistas con una referencia calculada a partir de las consultas HTTP.

12.2. Persistencia

La verificación de persistencia conserva las respuestas del estado de importación y de una venta pagada, reinicia SQL Server y vuelve a consultarlas. Se comparan ID, estado, total vendido, total abonado, saldo y hash del histórico. El registro guardado confirma que se conservaron después del reinicio. Esta operación utiliza el volumen existente y no requiere cargar nuevamente los datos.

12.3. Diagnóstico operativo

Si Postman no puede conectarse, se revisan la terminal de Node, baseUrl y el puerto de la API. Si falla la autenticación SQL, se comprueban salud del motor, salida de db-init y coincidencia de las credenciales con el volumen. Un db-init terminado con código 0 es normal; cualquier otro código requiere revisar su log antes de operar.

Un 409 de duplicado exige revisar el ID utilizado; uno por referencias exige tratar los registros asociados en el orden del negocio. Un 400 de inventario se verifica consultando el catálogo de la tienda y producto de la venta. Una ruta de importación deshabilitada devuelve 404 de forma prevista. Ninguno de esos casos se corrige borrando el volumen o desactivando restricciones.

12.4. Inspección y resguardo

DBeaver se conecta a ComercialLaEstrella con los parámetros locales y permite revisar el esquema dbo, sus restricciones, vistas y triggers. Las modificaciones de negocio se realizan mediante la API. El modo de solo lectura de la herramienta no sustituye los permisos SQL del usuario utilizado.

Las credenciales se conservan en .env fuera del control de versiones. El acceso HTTP y SQL se mantiene local; el backend no implementa autenticación general para publicación en Internet. La entrega se prepara con el código, el DDL, Compose, datos, colecciones y documentación, excluyendo secretos y dependencias instaladas.

A. Implementación de las consultas y controles transaccionales

Los fragmentos de este anexo corresponden a las vistas y triggers que ejecuta la instalación descrita. Sus reglas se aplican en el motor cuando la API escribe o consulta los registros; no requieren ejecución manual de sentencias durante la operación del cliente.

A.1. Vista_Ventas_Ubicacion

La agregación previa por venta permite contar encabezados sin multiplicarlos por sus detalles. El filtro de estado se aplica antes de agrupar la facturación por establecimiento.

```
CREATE OR ALTER VIEW dbo.Vista_Ventas_Ubicacion AS
WITH totales AS (
    SELECT id_venta, SUM(subtotal) AS total FROM dbo.detalle_venta GROUP BY id_venta
)
SELECT t.id_tienda, t.nombre AS tienda, m.nombre AS municipio,
       d.nombre AS departamento, p.nombre AS pais,
       COUNT_BIG(v.id_venta) AS cantidad_ventas,
       COALESCE(SUM(tot.total), 0) AS total_facturado
FROM dbo.venta v
JOIN dbo.estado_venta ev ON ev.id_estado_venta = v.id_estado_venta
JOIN dbo.tienda t ON t.id_tienda = v.id_tienda
JOIN dbo.municipio m ON m.id_municipio = t.id_municipio
JOIN dbo.departamento d ON d.id_departamento = m.id_departamento
JOIN dbo.pais p ON p.id_pais = d.id_pais
LEFT JOIN totales tot ON tot.id_venta = v.id_venta
WHERE ev.nombre <> N'ANULADA'
GROUP BY t.id_tienda, t.nombre, m.nombre, d.nombre, p.nombre;
```

A.2. Vista_Top_Productos

La agrupación utiliza el identificador del producto junto con sus dimensiones descriptivas; dos productos con el mismo nombre no se fusionan. Las ventas anuladas quedan excluidas.

```
CREATE OR ALTER VIEW dbo.Vista_Top_Productos AS
SELECT p.id_producto AS codigo_producto, p.nombre, c.nombre AS categoria,
       m.nombre AS marca, SUM(CONVERT(BIGINT,d.cantidad)) AS unidades_vendidas,
       SUM(d.subtotal) AS monto_total
FROM dbo.detalle_venta d
JOIN dbo.venta v ON v.id_venta = d.id_venta
JOIN dbo.estado_venta ev ON ev.id_estado_venta = v.id_estado_venta
JOIN dbo.producto p ON p.id_producto = d.id_producto
JOIN dbo.categoria c ON c.id_categoria = p.id_categoria
JOIN dbo.marca m ON m.id_marca = p.id_marca
WHERE ev.nombre <> N'ANULADA'
GROUP BY p.id_producto, p.nombre, c.nombre, m.nombre;
```

A.3. Vista_Consulta_Ventas

Los detalles y pagos se reducen a una fila por venta antes de unirlos. Los LEFT JOIN conservan ventas que todavía no tienen movimientos y COALESCE presenta sus importes como cero.

```
CREATE OR ALTER VIEW dbo.Vista_Consulta_Ventas AS
WITH totales AS (
    SELECT id_venta, SUM(subtotal) AS total FROM dbo.detalle_venta GROUP BY id_venta
), pagos AS (
    SELECT id_venta, SUM(monto) AS pagado FROM dbo.pago GROUP BY id_venta
)
SELECT v.*, e.nombre AS estado, COALESCE(t.total,0) AS total_venta,
       COALESCE(p.pagado,0) AS total_pagado,
       COALESCE(t.total,0)-COALESCE(p.pagado,0) AS saldo_pendiente
FROM dbo.venta v
JOIN dbo.estado_venta e ON e.id_estado_venta = v.id_estado_venta
LEFT JOIN totales t ON t.id_venta = v.id_venta
LEFT JOIN pagos p ON p.id_venta = v.id_venta;
```

A.4. Vista_Saldos_Pendientes

La selección reutiliza los importes consolidados y limita el resultado a REGISTRADA. No vuelve a calcular pagos ni modifica el estado al consultar.

```
CREATE OR ALTER VIEW dbo.Vista_Saldos_Pendientes AS
SELECT id_venta, total_venta, total_pagado, saldo_pendiente
FROM dbo.Vista_Consulta_Ventas WHERE estado = N'REGISTRADA';
```

A.5. Insert_Detalle

El conjunto de ventas se bloquea antes del ajuste. El delta agrupa inserted y deleted, por lo que INSERT, UPDATE y DELETE utilizan la misma regla. La comparación con @@ROWCOUNT detecta catálogos ausentes o disponibilidad insuficiente y provoca el rechazo de toda la sentencia.

```
CREATE OR ALTER TRIGGER dbo.Insert_Detalle ON dbo.detalle_venta
AFTER INSERT, UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    IF IS_ROLEMEMBER(N'importador') = 1
       AND TRY_CONVERT(INT, SESSION_CONTEXT(N'importacion_historica')) = 1 RETURN;

    DECLARE @ventas TABLE (id INT PRIMARY KEY);
    INSERT @ventas SELECT id_venta FROM inserted UNION SELECT id_venta FROM deleted;
    DECLARE @lock INT;
    SELECT @lock = v.id_venta FROM dbo.venta v WITH (UPDLOCK, HOLDLOCK)
    JOIN @ventas a ON a.id = v.id_venta ORDER BY v.id_venta;
    IF EXISTS (
        SELECT 1 FROM @ventas a JOIN dbo.venta v ON v.id_venta=a.id
        JOIN dbo.estado_venta e ON e.id_estado_venta=v.id_estado_venta
        WHERE e.nombre <> N'REGISTRADA' OR EXISTS(SELECT 1 FROM dbo.pago p WHERE p.id_venta=a.id)
    ) THROW 51002, N'Solo se permiten detalles en ventas REGISTRADA sin pagos', 1;

    DECLARE @delta TABLE (id_tienda INT, id_producto INT, cantidad BIGINT, PRIMARY KEY(id_tienda,id_producto));
    INSERT @delta
    SELECT v.id_tienda, x.id_producto, SUM(x.cantidad)
    FROM (
        SELECT id_venta,id_producto,CONVERT(BIGINT,cantidad) cantidad FROM inserted
        UNION ALL
        SELECT id_venta,id_producto,-CONVERT(BIGINT,cantidad) FROM deleted
    ) x JOIN dbo.venta v ON v.id_venta=x.id_venta
    GROUP BY v.id_tienda,x.id_producto;

    UPDATE c WITH (UPDLOCK, HOLDLOCK)
       SET existencia_actual = c.existencia_actual - d.cantidad
    FROM dbo.catalogo_producto c JOIN @delta d
      ON d.id_tienda=c.id_tienda AND d.id_producto=c.id_producto
    WHERE CONVERT(BIGINT,c.existencia_actual) >= d.cantidad;
    IF @@ROWCOUNT <> (SELECT COUNT(*) FROM @delta)
        THROW 51001, N'Stock insuficiente', 1;
END;
```

A.6. Insert_Pago

La comprobación de venta anulada y de detalle previo precede al recálculo. La condición de cobertura utiliza la suma vigente después de insertar, modificar o eliminar pagos. La actualización del encabezado se origina dentro del motor.

```
CREATE OR ALTER TRIGGER dbo.Insert_Pago ON dbo.pago
AFTER INSERT, UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    IF IS_ROLEMEMBER(N'importador') = 1
       AND TRY_CONVERT(INT, SESSION_CONTEXT(N'importacion_historica')) = 1 RETURN;
    DECLARE @ventas TABLE (id INT PRIMARY KEY);
    INSERT @ventas SELECT id_venta FROM inserted UNION SELECT id_venta FROM deleted;
    DECLARE @lock INT;
    SELECT @lock=v.id_venta FROM dbo.venta v WITH(UPDLOCK,HOLDLOCK)
    JOIN @ventas a ON a.id=v.id_venta ORDER BY v.id_venta;
    IF EXISTS (
        SELECT 1 FROM @ventas a JOIN dbo.venta v ON v.id_venta=a.id
        JOIN dbo.estado_venta e ON e.id_estado_venta=v.id_estado_venta
        WHERE e.nombre=N'ANULADA'
    ) THROW 51002, N'No se permiten pagos en ventas anuladas', 1;
    IF EXISTS (
        SELECT 1 FROM inserted i WHERE NOT EXISTS (
            SELECT 1 FROM dbo.detalle_venta d WHERE d.id_venta=i.id_venta
        )
    ) THROW 51002, N'La venta debe tener detalles antes de recibir pagos', 1;

    UPDATE v SET id_estado_venta = e.id_estado_venta
    FROM dbo.venta v JOIN @ventas a ON a.id=v.id_venta
    CROSS APPLY (SELECT COALESCE(SUM(subtotal),0) total FROM dbo.detalle_venta WHERE id_venta=v.id_venta) d
    CROSS APPLY (SELECT COALESCE(SUM(monto),0) total FROM dbo.pago WHERE id_venta=v.id_venta) p
    JOIN dbo.estado_venta e ON e.nombre = CASE WHEN d.total > 0 AND p.total >= d.total THEN N'PAGADA' ELSE N'REGISTRADA'
     END;
END;
```

A.7. Validar_Venta

La validación distingue una transición automática desde Insert_Pago de una edición directa. Impide trasladar movimientos a otros datos de encabezado y devuelve inventario solamente cuando se pasa de REGISTRADA a ANULADA sin pagos.

```
CREATE OR ALTER TRIGGER dbo.Validar_Venta ON dbo.venta
AFTER INSERT, UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    IF IS_ROLEMEMBER(N'importador') = 1
       AND TRY_CONVERT(INT, SESSION_CONTEXT(N'importacion_historica')) = 1 RETURN;
    IF TRIGGER_NESTLEVEL(OBJECT_ID(N'dbo.Insert_Pago')) > 0 RETURN;
    IF EXISTS (
        SELECT 1 FROM inserted i LEFT JOIN deleted d ON d.id_venta=i.id_venta
        JOIN dbo.estado_venta e ON e.id_estado_venta=i.id_estado_venta
        WHERE d.id_venta IS NULL AND e.nombre <> N'REGISTRADA'
    ) THROW 51002,N'Una venta nueva debe iniciar REGISTRADA',1;
    IF EXISTS (
        SELECT 1 FROM inserted i JOIN deleted d ON d.id_venta=i.id_venta
        JOIN dbo.estado_venta anterior ON anterior.id_estado_venta=d.id_estado_venta
        JOIN dbo.estado_venta nuevo ON nuevo.id_estado_venta=i.id_estado_venta
        WHERE (anterior.nombre=N'ANULADA')
           OR (i.id_estado_venta<>d.id_estado_venta AND NOT (
                 anterior.nombre=N'REGISTRADA' AND nuevo.nombre=N'ANULADA'
                 AND NOT EXISTS(SELECT 1 FROM dbo.pago p WHERE p.id_venta=i.id_venta)))
           OR ((i.id_tienda<>d.id_tienda OR i.id_empleado<>d.id_empleado OR i.id_cliente<>d.id_cliente OR i.fecha_venta<>d.
     fecha_venta)
               AND (EXISTS(SELECT 1 FROM dbo.detalle_venta dv WHERE dv.id_venta=i.id_venta)
                 OR EXISTS(SELECT 1 FROM dbo.pago p WHERE p.id_venta=i.id_venta)))
    ) THROW 51002,N'Cambio de venta no permitido: estado automatico, venta anulada o registros asociados',1;

    -- Devolver stock solo en la transicion REGISTRADA -> ANULADA.
    ;WITH devolucion AS (
        SELECT i.id_tienda,dv.id_producto,SUM(CONVERT(BIGINT,dv.cantidad)) cantidad
        FROM inserted i JOIN deleted d ON d.id_venta=i.id_venta
        JOIN dbo.estado_venta e ON e.id_estado_venta=i.id_estado_venta
        JOIN dbo.detalle_venta dv ON dv.id_venta=i.id_venta
        WHERE i.id_estado_venta<>d.id_estado_venta AND e.nombre=N'ANULADA'
        GROUP BY i.id_tienda,dv.id_producto
    )
    UPDATE c SET existencia_actual=c.existencia_actual+x.cantidad
    FROM dbo.catalogo_producto c JOIN devolucion x ON x.id_tienda=c.id_tienda AND x.id_producto=c.id_producto;
END;
```

A.8. Proteger_Catalogo

La validación busca claves presentes en deleted que ya no están en inserted y comprueba si dan contexto a detalles. Así protege la desaparición de la asociación sin impedir el ajuste legítimo de existencias.

```
CREATE OR ALTER TRIGGER dbo.Proteger_Catalogo ON dbo.catalogo_producto
AFTER UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM deleted d
        WHERE NOT EXISTS (SELECT 1 FROM inserted i WHERE i.id_tienda=d.id_tienda AND i.id_producto=d.id_producto)
          AND EXISTS (SELECT 1 FROM dbo.detalle_venta dv JOIN dbo.venta v ON v.id_venta=dv.id_venta
                      WHERE v.id_tienda=d.id_tienda AND dv.id_producto=d.id_producto)
    ) THROW 51002,N'El catalogo tiene detalles de venta asociados',1;
END;
```

A.9. Proteger_Estados

El control conserva los IDs y nombres que utilizan las operaciones y el histórico. Las pruebas de escritura sobre este catálogo deben terminar en conflicto y no en la creación de un cuarto estado.

```
CREATE OR ALTER TRIGGER dbo.Proteger_Estados ON dbo.estado_venta
AFTER INSERT, UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS(SELECT 1 FROM deleted) THROW 51002,N'Los estados del sistema no se modifican ni eliminan',1;
    IF EXISTS(SELECT 1 FROM inserted WHERE (id_estado_venta=1 AND nombre<>N'REGISTRADA')
       OR (id_estado_venta=2 AND nombre<>N'PAGADA') OR (id_estado_venta=3 AND nombre<>N'ANULADA')
       OR id_estado_venta NOT IN(1,2,3)) THROW 51002,N'Estado del sistema invalido',1;
END;
```

B. Referencias de la implementación

- Enunciado ProyectoFase2_SBD1_2S2026 y hoja de calificación de la fase 2.
- Modelo y dataset de Comercial La Estrella de la fase 1.
- Código y colecciones del repositorio: https://github.com/ricardious/SBD1_2S2026.
- Registro de operaciones y capturas directas de la API, con identificador de escenario, fecha UTC y hash de las imágenes.